

Disk Space Investigation

Issue: Drive shows 95-100% full but visible files don't account for the usage.

Decision Tree

```
Disk nearly full but usage doesn't add up
■■■ Get actual vs. reported usage
■   ■■■ Right-click C: → Properties → note total used
■   ■■■ Select all folders in C:\ → Properties → does the sum match?
■   ■   ■ NO (big gap) → Hidden consumers → INVESTIGATION
■   ■   ■ YES → Disk is legitimately full → CLEANUP
```

Investigation: Hidden Space Consumers

Run these checks in order.

1. WinSxS Component Store

```
dism /Online /Cleanup-Image /AnalyzeComponentStore
```

Explorer inflates WinSxS size due to hard links. DISM shows the real footprint. If cleanup is recommended:

```
dism /Online /Cleanup-Image /StartComponentCleanup
```

2. Hibernation File

PowerShell:

```
Get-Item C:\hiberfil.sys -Force -ErrorAction SilentlyContinue | Select-Object Length
```

CMD:

```
dir /a C:\hiberfil.sys
```

Typically 40-75% of installed RAM. Remove with `powercfg /h off` (works in both shells).

3. Page File

PowerShell:

```
Get-Item C:\pagefile.sys -Force -ErrorAction SilentlyContinue | Select-Object Length
```

CMD:

```
dir /a C:\pagefile.sys
```

Don't delete. Reduce via System Properties → Performance → Virtual Memory if oversized.

4. User Profiles

CMD has no built-in recursive folder-size command. For a quick per-folder size in

CMD, use: `dir /s "C:\Users\<name>"` and read the final "bytes" total, or install the Sysinternals `du` tool: `du -l 1 C:\Users`. The PowerShell loop below is the cleanest native option.

```
Get-ChildItem C:\Users -Directory | ForEach-Object {
    $size = (Get-ChildItem $_.FullName -Recurse -Force -ErrorAction SilentlyContinue |
        Measure-Object Length -Sum).Sum
    [PSCustomObject]@{ User = $_.Name; SizeGB = [math]::Round($size / 1GB, 2) }
} | Sort-Object SizeGB -Descending | Format-Table -AutoSize
```

5. Previous Windows Installations

```
@("C:\Windows.old", "C:\$WINDOWS.~BT", "C:\$WINDOWS.~WS") | ForEach-Object {
    if (Test-Path $_) {
        $size = (Get-ChildItem $_ -Recurse -Force -ErrorAction SilentlyContinue |
            Measure-Object Length -Sum).Sum
        Write-Host "$_ : $([math]::Round($size / 1GB, 2)) GB"
    }
}
```

Remove via Disk Cleanup → System files → Previous Windows installations.

6. Shadow Copies

```
vssadmin list shadowstorage
vssadmin resize shadowstorage /for=C: /on=C: /maxsize=5GB
```

7. Deployment Tool Caches

```
# SCCM cache
if (Test-Path "C:\Windows\ccmcache") {
    $size = (Get-ChildItem "C:\Windows\ccmcache" -Recurse -Force -ErrorAction SilentlyContinue |
        Measure-Object Length -Sum).Sum
    Write-Host "SCCM Cache: $([math]::Round($size / 1GB, 2)) GB"
}
```

8. Full Folder Scan

```
Get-ChildItem C:\ -Directory -Force -ErrorAction SilentlyContinue | ForEach-Object {
    $size = (Get-ChildItem $_.FullName -Recurse -Force -ErrorAction SilentlyContinue |
        Measure-Object Length -Sum -ErrorAction SilentlyContinue).Sum
    [PSCustomObject]@{ Folder = $_.Name; SizeGB = [math]::Round($size / 1GB, 2) }
} | Sort-Object SizeGB -Descending | Format-Table -AutoSize
```

Cleanup via Orchestrator

```
# Safe first pass
.\Invoke-HelpDeskOrchestrator.ps1 -RelieveDiskPressure -CleanupTemp -DisableHibernation

# If still tight and endpoint is stable (IRREVERSIBLE)
.\Invoke-HelpDeskOrchestrator.ps1 -RelieveDiskPressure -RunComponentCleanup -AllowIrreversible
```

Common Recovery Amounts

Target	Typical Recovery
Temp files	0.5-5 GB
Windows.old	10-25 GB
Hibernation	Equal to RAM
WinSxS cleanup	1-5 GB
Delivery Optimization cache	1-5 GB
SCCM/MEC cache	2-25 GB

Windows Endpoint Remediation Toolkit — Author: Adrian Melendez

Windows Endpoint Remediation Toolkit